

NVIDIA / Isaac-GR00T

Code

Issues107

Pull requests96

Agents

Actions

Projects

Security and quality

Insights

Watch76

NVIDIA Isaac GR00T N1.7 - A Foundation Model for Generalist Robots.

developer.nvidia.com/isaac/gr00t

Apache-2.0 license

Contributing

7.7k stars

1.3k forks

76 watching

Branches

Activity

Custom properties

Tags

Public repository

main59 Branches5 Tags

Go to fileTGo to fileAdd fileCode

SyncBot and ryhalabi - SimplexEnv and Robocasa setup script fixes. 9c7e746 · 2 weeks ago		
.github	Isaac GR00T N1.6 Release (#456)	3 months ago
docs	distributed training	1 month ago
examples	General F	
isaac	ra	
isaac_ros	ra	
isaacsim	and Rob	
models	Release	1 month ago
scripts	Isaac GR00T N1.7 General Release	2 weeks ago
tests	GR00T N1.7 General Release	2 weeks ago
tools	Bug fixes and expanded 1.7 metrics/benchm...	2 months ago
.coveragerc	GR00T N1.7 Release	3 months ago
.dockerignore	GR00T N1.7 General Release	2 weeks ago
.gitattributes	GR00T N1.7 General Release	2 weeks ago
.gitignore	- Add the N1.7 / SONIC G1 real-world Whol...	last month
.gitmodules	Bug fixes and expanded 1.7 metrics/benchm...	2 months ago
.pre-commit-config.yaml	- Adds shared deployment mode enums in ...	last month
AGENTS.md	GR00T N1.7 Release	3 months ago
ATTRIBUTIONS.md	GR00T N1.7 General Release	2 weeks ago
CLAUDE.md	GR00T N1.7 General Release	2 weeks ago
CONTRIBUTING.md	GR00T N1.7 General Release	2 weeks ago
FAQ.md	GR00T N1.7 General Release	2 weeks ago
LICENSE	GR00T N1.7 Release	3 months ago
README.md	GR00T N1.7 General Release	2 weeks ago
pyproject.toml	GR00T N1.7 General Release	2 weeks ago
uv.lock	GR00T N1.7 General Release	2 weeks ago



Isaac GR00T

[Website](#) | [Model](#) | [Datasets \(Physical AI\)](#) | [Paper](#) | [NVIDIA Isaac](#) | [FAQ](#)

Table of Contents

- [NVIDIA Isaac GR00T](#)
- [What's New in GR00T N1.7](#)
- [Installation](#)
- [LeRobot Integration](#)
- [Model Checkpoints & Embodiment Tags](#)
- [Data Format](#)
- [Inference](#)
- [Fine-tuning](#)
- [Evaluation](#)
- [Contributions](#)
- [License](#)
- [Citation](#)

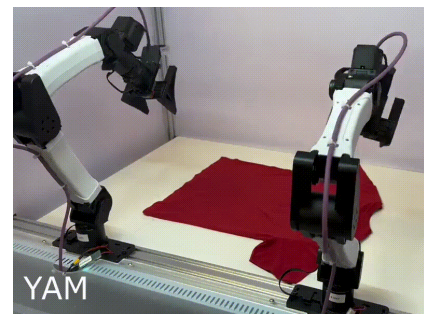
NVIDIA Isaac GR00T



Unitree G1



AgiBot G1



YAM

We just released GR00T N1.7 General Availability, the latest version of GR00T N1 with a new VLM backbone (Cosmos-Reason2-2B / Qwen3-VL) and improved performance.

This is a General Availability (GA) release. You are welcome to download the model, explore the codebase, and build on the stack, with full support and stability guarantees.

What's available:

- Pre-trained GR00T N1.7 model weights and reference code
- Fine-tuning and inference with custom robot data or demonstrations
- Experimentation, prototyping, and research use cases
- Production deployment with commercial support
- Complete benchmarks and a fully validated, stable feature set
- Pull request contributions

We welcome feedback - please feel free to raise issues and pull requests in this repository.

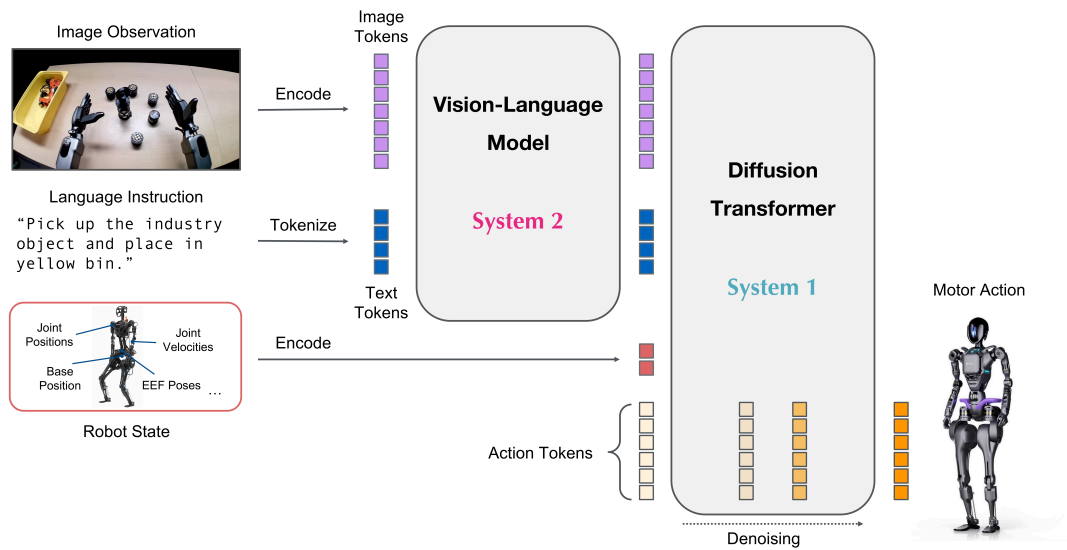
Previous releases: [N1.6](#) | [N1.5](#)

NVIDIA Isaac GR00T N1.7 is an open vision-language-action (VLA) model for generalized humanoid robot skills. This cross-embodiment model takes multimodal input, including language and images, to perform manipulation tasks in diverse environments.

GR00T N1.7 is trained on a diverse mixture of robot data including bimanual, semi-humanoid and an expansive humanoid dataset. It is adaptable through post-training for specific embodiments, tasks and environments.

GR00T N1.7 is fully commercially licensable under Apache 2.0. It delivers comparable performance to N1.6, with improved generalization and language-following capabilities driven by the inclusion of 20K hours of EgoScale human video data in pretraining.

The neural network architecture of GR00T N1.7 is a combination of vision-language foundation model and diffusion transformer head that denoises continuous actions. Here is a schematic diagram of the architecture:



Workflow Overview

1. **Prepare data** — Collect robot demonstrations (video, state, action) and convert them to the [GR00T LeRobot format](#). Demo datasets are included for quick testing.
2. **Run inference** — Try zero-shot inference with the base model on [pretrain embodiments](#), or use a [finetuned checkpoint](#) for benchmark tasks.
3. **Fine-tune** — Adapt the model to your robot using [launch finetune.py](#) with your own data and modality config.
4. **Evaluate** — Validate with [open-loop evaluation](#), then test in [simulation benchmarks](#) or on real hardware via the [Policy API](#).
5. **Deploy** — Connect `gr00tPolicy` to your robot controller, optionally accelerated with [TensorRT](#).

What's New in GR00T N1.7

GR00T N1.7 builds on N1.6 with a new VLM backbone and code-level improvements.

1. **Relative EEf Action Space** — N1.7 adopts a relative end-effector action space shared across robot and human embodiments. Representing actions as deltas from the current pose (rather than absolute targets) improves generalization and is a key factor in the model's cross-embodiment performance. See [getting_started/finetune_new_embodiment.md](#) for guidance on configuring relative EEf for your own robot.
2. **Human Video Pretraining** — N1.7 is pretrained on 20K hours of EgoScale human video data alongside diverse robot demonstrations. Because the relative EEf action representation is consistent across both human and robot data, the model can transfer manipulation prior learned from human video directly to robot control.

Key Changes from N1.6

Compared with N1.6, N1.7 updates the model stack, training data interface, evaluation coverage, deployment flow, fine-tuning workflow, and runtime behavior.

- **New VLM backbone:** Cosmos-Reason2-2B (Qwen3-VL architecture), replacing the Eagle backbone used in N1.6. Supports flexible resolution and encodes images in their native aspect ratio without padding.
- **Updated model interface:** N1.7 moves to the `gr00t_n1d7` model package, expands the state/action dimensions, and increases the model action horizon.
- **More flexible dataset handling:** Fine-tuning can use multiple dataset paths with mixture weighting, making multi-dataset training easier to configure.
- **Broader benchmark coverage:** N1.7 refreshes and expands documented results across RoboCasa, RoboCasa GR1 tabletop tasks, SimplerEnv, and real G1 evaluation.
- **More complete deployment path:** N1.7 adds full-pipeline ONNX and TensorRT export support and improves deployment consistency across desktop GPUs and edge platforms.

- **More predictable runtime behavior:** Policy serving, rollout recording, evaluation, and configuration validation have been hardened so errors are easier to diagnose.

► Detailed changes from N1.6

Installation

Hardware Requirements

Inference: 1 GPU with 16 GB+ VRAM (e.g., RTX 4090, L40, H100, Jetson AGX Thor/Orin, DGX Spark).

Fine-tuning: 1 or more GPUs with 40 GB+ VRAM recommended. We recommend H100 or L40 nodes for optimal performance. Other hardware (e.g., A6000) works but may require longer training time. See the [Hardware Recommendation Guide](#) for detailed specs.

CUDA / Python per platform: dGPU on CUDA 12.8 with Python 3.12; Jetson Orin on CUDA 12.6 with Python 3.10; Jetson Thor and DGX Spark on CUDA 13.0 with Python 3.12. The per-platform install scripts and Dockerfiles live under `scripts/deployment/`; see the [Deployment & Inference Guide](#) for the full matrix.

Clone the Repository

GR00T relies on submodules for certain dependencies. Include them when cloning:

Note: `git-lfs` is required to download parquet data files in `demo_data/`. Install it before cloning: `sudo apt install git-lfs && git lfs install`.

```
git clone --recurse-submodules https://github.com/NVIDIA/Isaac-GR00T
cd Isaac-GR00T
```



If you've already cloned without submodules, initialize them separately:

```
git submodule update --init --recursive
```



Set Up the Environment

GR00T uses [uv](#) for fast, reproducible dependency management. Install uv first:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```



dGPU (x86_64) — Default

Install FFmpeg (required by `torchcodec`, the only supported video backend):

```
sudo apt-get update && sudo apt-get install -y ffmpeg
```



FFmpeg version: `torchcodec==0.8.0` supports **FFmpeg 4-7 only**. On Ubuntu 25.10+/26.04 the `ffmpeg` package is version 8, which `torchcodec` cannot load (`RuntimeError: Could not load libtorchcodec ... We support versions 4, 5, 6 and 7`). On those distros install an `FFmpeg<8` runtime instead, e.g. `conda install -c conda-forge 'ffmpeg<8'`, and make sure its libraries are on `LD_LIBRARY_PATH`.

Create the environment and install GR00T:

```
uv sync --python 3.12
```



GPU dependencies (flash-attn, TensorRT, etc.) are included in the default install.

Verify the installation:

```
uv run python -c "import gr00t; print('GR00T installed successfully')"
```



Hugging Face access (required): GR00T's VLM backbone is [nvidia/Cosmos-Reason2-2B](#), a **gated** model that every GR00T checkpoint (including the base `nvidia/GR00T-N1.7-3B`) loads on first use. Before running inference or finetuning, request access on the model page and authenticate:

```
uv run hf auth login # or: export HF_TOKEN=<your_token>
```



Without access, model loading fails with a `GatedRepoError / 401 Client Error`.

flash-attn message on every `uv run`: You may see `Installing flash-attn...` each time you run `uv run`. This is a known `uv` behavior with URL-pinned wheel sources — `uv` re-validates the cached wheel against the source URL on each invocation. It is **not** rebuilding from source; the wheel is already cached locally and the operation takes 2-3 seconds. This affects platforms that use URL-pinned flash-attn wheels (x86_64 and aarch64). To suppress it, remove the `flash-attn` entries under `[tool.uv.sources]` in your local `pyproject.toml` after the initial install. But that will break `uv lock` and cause flash-attn to build from source on next lock regeneration.

► Alternative: pip install (without uv)

If fine-tuning fails with `CUDA_HOME` is unset: Run `bash scripts/deployment/dgpu/install_deps.sh` once to configure CUDA paths, or manually `export CUDA_HOME=/usr/local/cuda`.

CUDA 13.x Users (Thor, Spark, and other CUDA 13+ platforms): PyTorch 2.7 pins Triton to 3.3.1, which does not recognize CUDA major version 13+. This causes a `RuntimeError` in Triton's `ptx_get_version()`. Run `scripts/patch_triton_cuda13.sh` to fix:

```
uv run bash scripts/patch_triton_cuda13.sh
```



GB300 (sm_103) Users: Triton 3.3.1 (pinned by PyTorch 2.7) does not support the GB300 GPU architecture (sm_103). `torch.compile` will fail on GB300. Use PyTorch eager mode or TensorRT inference instead. Triton 3.5.1+ adds sm_103 support but is not yet compatible with the pinned PyTorch version.

Video Backend: GR00T uses [torchcodec](#) as its sole video decoding backend. Backends such as `decord` and `pyav` are no longer supported. `torchcodec` 0.8.0 requires **FFmpeg 4-7** (FFmpeg 8 is not supported — see the FFmpeg version note above) and supports H.264 on all platforms; AV1 decoding is not guaranteed (convert AV1 datasets to H.264 with `examples/SimplerEnv/convert_av1_to_h264.py`). On aarch64 platforms (Thor, Orin), `torchcodec` is built from source during `install_deps.sh` because pre-built wheels are not available — if you encounter a `NotImplementedError`, ensure the build completed successfully.

► DGX Spark (tested with DGX Spark GB10)

► Jetson AGX Thor (tested with JetPack 7.1)

► Jetson Orin (tested with JetPack 6.2)

⚠️ aarch64 users (Spark / Thor / Orin): After running `install_deps.sh`, always activate the venv with `source .venv/bin/activate && source scripts/activate_<platform>.sh` (`activate_spark.sh`, `activate_thor.sh`, or `activate_orin.sh`) and run the example commands in this guide with **plain python** / `torchrun`, not `uv run python` / `uv run torchrun`. The latter will re-sync against the root `pyproject.toml` (which targets x86_64 Python 3.12) and destroy the platform-specific environment. See the [Deployment & Inference Guide](#) for per-platform Docker and bare-metal setup.

For a containerized setup that avoids system-level dependency conflicts, see our [Docker Setup Guide](#). The recommended container workflow to start the image first, then clone or pull the repo inside the running container so your checkout uses the image's prebuilt dependency environment.

LeRobot Integration

GR00T N1.7 is also available through Hugging Face LeRobot via the `groot` policy type. Use the [LeRobot GR00T documentation](#) for LeRobot-native training, evaluation, and rollout workflows. Use this repository for the reference GR00T implementation, model internals, deployment tooling, and benchmark-specific examples.

Model Checkpoints & Embodiment Tags

Checkpoints

Checkpoint	Type	Embodiment Tag	Description
nvidia/GR00T-N1.7-3B	Base	See pretrain tags	Base model (3B params) — zero-shot inference on pretrain embodiments, or finetune for new tasks
nvidia/GR00T-N1.7-LIBERO	Finetuned	LIBERO_PANDA	Finetuned on LIBERO benchmark (Franka Panda)

Checkpoint	Type	Embodiment Tag	Description
nvidia/GR00T-N1.7-DROID	Finetuned	OXE_DROID_RELATIVE_EEF_RELATIVE_JOINT	Finetuned on DROID dataset
nvidia/GR00T-N1.7-SimplerEnv-Bridge	Finetuned	SIMPLER_ENV_WIDOWX	Finetuned on SimplerEnv Bridge (WidowX)
nvidia/GR00T-N1.7-SimplerEnv-Fractal	Finetuned	SIMPLER_ENV_GOOGLE	Finetuned on SimplerEnv Fractal (Google Robot)

Embodiment Tags

Every inference or finetuning command requires an `--embodiment-tag` . The tag determines which modality config (state/action keys, normalization) the model uses. Tags are **case-insensitive**.

For the full list of pretrain and posttrain tags, see the [Policy API Guide — Embodiment Tags](#).

Data Format

GR00T uses a flavor of the [LeRobot v2 dataset format](#) with an additional `meta/modality.json` file that describes state/action/video structure. A dataset looks like:

```
my_dataset/
  meta/
    info.json           # dataset metadata
    episodes.jsonl      # episode index and lengths
    tasks.jsonl         # language task descriptions
    modality.json       # state/action/video key mapping (GR00T-specific)
  data/chunk-000/       # parquet files (state, action per timestep)
  videos/chunk-000/     # mp4 video files per episode
```

The `modality.json` maps how the concatenated state/action arrays split into named fields (e.g., `x` , `y` , `z` , `gripper`) and which video keys are available. This is what the embodiment tag uses to interpret the data.

Included demo datasets (ready to use, no download needed):

Dataset	Robot	Embodiment Tag	Use Case
demo_data/droid_sample	DROID (3 episodes)	OXE_DROID_RELATIVE_EEF_RELATIVE_JOINT	Zero-shot or finetuned inference (DROID)
demo_data/libero_demo	LIBERO Panda (5 episodes)	LIBERO_PANDA	Inference with finetuned checkpoint
demo_data/simplerenv_bridge_sample	WidowX (SimplerEnv Bridge)	SIMPLER_ENV_WIDOWX	Inference with finetuned SimplerEnv Bridge checkpoint
demo_data/simplerenv_fractal_sample	Google Robot (SimplerEnv Fractal)	SIMPLER_ENV_GOOGLE	Inference with finetuned SimplerEnv Fractal checkpoint
demo_data/cube_to_bowl_5	SO100 arm (5 episodes)	NEW_EMBODIMENT	Fine-tuning custom embodiment example
demo_data/cube_to_bowl_5_with_mask	SO100 arm + per-frame masks	NEW_EMBODIMENT	Mask-guided background suppression example

| To generate more DROID episodes: `python scripts/download_droid_sample.py --num-episodes 10`

Using your own data: Convert your demonstrations to the format above. If coming from LeRobot v3, use the conversion helper in its own environment:

```
cd scripts/lerobot_conversion
uv venv
source .venv/bin/activate
uv pip install -e . --verbose
python convert_v3_to_v2.py --repo-id <DATASET_REPO_ID>
```

See the full [Data Preparation Guide](#) for schema details and examples.

Inference

Prefer an interactive walkthrough? The [getting_started/GR00T_inference.ipynb](#) notebook steps through loading the model and predicting actions from observations on a sample dataset.

Zero-Shot Inference (Base Model)

The included `demo_data/droid_sample` dataset works with the base model out of the box — no finetuning or checkpoint download needed:

```
uv run python scripts/deployment/standalone_inference_script.py \
  --model-path nvidia/GR00T-N1.7-3B \
  --dataset-path demo_data/droid_sample \
  --embodiment-tag OXE_DROID_RELATIVE_EEF_RELATIVE_JOINT \
  --traj-ids 1 2 \
  --inference-mode pytorch \
  --execution-horizon 8
```



This runs open-loop inference on 2 DROID episodes, comparing predicted actions against ground truth. The base model downloads automatically from HuggingFace on first run (~6 GB).

Note: The base model loads the gated `nvidia/Cosmos-Reason2-2B` backbone, so this command requires Hugging Face access (see [Set Up the Environment](#)). Without it the run fails with a `GatedRepoError`.

Finetuned Inference

For posttrain embodiments, use a finetuned checkpoint. Most finetuned checkpoints (e.g., DROID, SimplerEnv) have a flat file structure and can be passed directly as a HuggingFace model ID — no manual download needed:

```
uv run python scripts/deployment/standalone_inference_script.py \
  --model-path nvidia/GR00T-N1.7-DROID \
  --dataset-path demo_data/droid_sample \
  --embodiment-tag OXE_DROID_RELATIVE_EEF_RELATIVE_JOINT \
  --traj-ids 1 2 \
  --inference-mode pytorch \
  --execution-horizon 8
```



Some checkpoints (e.g., LIBERO) use a nested folder structure with model files under a subfolder. HuggingFace does not support nested repo paths in `--model-path`, so you must download first:

```
uv run hf download nvidia/GR00T-N1.7-LIBERO \
  --include "libero_10/config.json" "libero_10/embodiment_id.json" \
  "libero_10/model-*.safetensors" "libero_10/model.safetensors.index.json" \
  "libero_10/processor_config.json" "libero_10/statistics.json" \
  --local-dir checkpoints/GR00T-N1.7-LIBERO
```



```
uv run python scripts/deployment/standalone_inference_script.py \
  --model-path checkpoints/GR00T-N1.7-LIBERO/libero_10 \
  --dataset-path demo_data/libero_demo \
  --embodiment-tag LIBERO_PANDA \
  --traj-ids 0 1 2 \
  --inference-mode pytorch \
  --execution-horizon 8
```



Server-Client Inference (for Deployment)

For real-world deployment or simulation evaluation, use the server-client architecture. The policy runs on a GPU server; a lightweight client sends observations and receives actions over ZMQ.

Terminal 1 — Start the policy server:

```
uv run python gr00t/eval/run_gr00t_server.py \
  --model-path nvidia/GR00T-N1.7-3B \
  --embodiment-tag OXE_DROID_RELATIVE_EEF_RELATIVE_JOINT \
  --device cuda:0
```



Terminal 2 — Run open-loop evaluation as a client:

```
uv run python gr00t/eval/open_loop_eval.py \
  --dataset-path demo_data/droid_sample \
  --embodiment-tag OXE_DROID_RELATIVE_EEF_RELATIVE_JOINT \
  --host 127.0.0.1 \
  --port 5555 \
  --traj-ids 1 2 \
  --execution-horizon 8
```



Tip: If you get `ZMQError: Address already in use`, the default port 5555 is occupied. Use `--port <other_port>`.

For connecting to a real robot (e.g., DROID hardware), see [examples/DROID/README.md](#). For faster inference with TensorRT, see the [Deployment & Inference Guide](#).

See the complete [Policy API Guide](#) for documentation on observation/action formats, batched inference, and troubleshooting.

Fine-tuning

Reproducing Benchmark Results

Each benchmark has a self-contained README with dataset download, finetune, and evaluation commands:

Benchmark	Embodiment	Guide
LIBERO	LIBERO_PANDA	examples/LIBERO/README.md
SimplerEnv (Fractal)	SIMPLER_ENV_GOOGLE	examples/SimplerEnv/README.md
SimplerEnv (Bridge)	SIMPLER_ENV_WIDOWX	examples/SimplerEnv/README.md
SO100	NEW_EMBODIMENT	examples/SO100/README.md

Humanoid Whole-Body Control (SONIC)

GR00T N1.7 supports whole-body humanoid control via the `UNITREE_G1_SONIC` embodiment tag and the [GEAR-SONIC](#) controller. In this workflow, the VLA predicts compact latent action tokens that a learned whole-body controller decodes into full-body joint commands — including legs, arms, and hands. A single policy produces language-conditioned, coordinated manipulation and locomotion end-to-end. SON supports whole-body coordination with precise hand and foot placements.

The complete collect → finetune → deploy workflow is documented in the [GR00T-WholeBodyControl repository](#):

- [Data collection](#) — VR teleoperation with SONIC for demonstration recording
- [VLA Workflow](#) — finetuning Isaac-GR00T N1.7 on collected data and deploying the policy
- [VLA Inference](#) — running the PolicyServer + SONIC decoder for real-time control

Note: The `UNITREE_G1` embodiment tag is compatible with the [decoupled WBC](#) controller, but the end-to-end collect-finetune-deploy workflow is only supported for GEAR-SONIC (`UNITREE_G1_SONIC`).

Fine-tune on Your Own Robot ("NEW_EMBODIMENT")

To finetune GR00T on your own robot data and configuration, follow the detailed tutorial at [getting_started/finetune_new_embodiment.md](#).

Ensure your input data follows the [GR00T LeRobot format](#), and specify your modality configuration via `--modality-config-path`.

Single GPU:

```
CUDA_VISIBLE_DEVICES=0 uv run python \
  gr00t/experiment/launch_finetune.py \
  --base-model-path nvidia/GR00T-N1.7-3B \
  --dataset-path demo_data/cube_to_bowl_5 \
  --embodiment-tag NEW_EMBODIMENT \
  --modality-config-path examples/SO100/so100_config.py \
  --num-gpus 1 \
  --output-dir /tmp/test_finetune \
  --max-steps 2000 \
  --global-batch-size 32 \
  --dataloader-num-workers 4
```



Multi-GPU (e.g., 8xH100):


```
uv run torchrun --nproc_per_node=8 --master_port=29500 \
  gr00t/experiment/launch_finetune.py \
  --base-model-path nvidia/GR00T-N1.7-3B \
  --dataset-path demo_data/cube_to_bowl_5 \
  --embodiment-tag NEW_EMBODIMENT \
  --modality-config-path examples/S0100/so100_config.py \
  --num-gpus 8 \
  --output-dir /tmp/test_finetune_8gpu \
  --max-steps 2000 \
  --global-batch-size 32 \
  --dataloader-num-workers 4
```



Replace `demo_data/cube_to_bowl_5` and `examples/S0100/so100_config.py` with your own dataset and modality config. See [examples/S0100](#) for a complete walkthrough.

Note: Use `uv run torchrun` (not bare `torchrun`) to ensure the correct virtual environment is used. Add `--use-wandb` to enable Weights & Biases logging. For more extensive configuration, use `gr00t/experiment/launch_train.py`.

Training Tips

- Maximize batch size for your hardware and train for a few thousand steps.
- Users may observe 5-6% variance between runs due to non-deterministic image augmentations. Keep this in mind when comparing to reported benchmarks.
- `--state_dropout_prob` (model config default: 0.8; finetune CLI default: 0.2; see `gr00t/configs/finetune_config.py`): Randomly drops state inputs during training to improve generalization and reduce state-dependency. The shipped benchmark scripts override the CLI default p suite: LIBERO 10-Long uses 0.2 (the CLI default), SimplerEnv Bridge uses 0.8, SimplerEnv Fractal uses 0.5. If your task relies heavily on proprioceptive state, lower this value.

Evaluation

Open-Loop Evaluation

Compare predicted actions against ground truth from your dataset:

```
uv run python gr00t/eval/open_loop_eval.py \
  --dataset-path <DATASET_PATH> \
  --embodiment-tag NEW_EMBODIMENT \
  --model-path <CHECKPOINT_PATH> \
  --traj-ids 0 \
  --execution-horizon 16
```



This generates a visualization at `/tmp/open_loop_eval/traj_{traj_id}.jpeg` with ground truth vs. predicted actions and MSE metrics. Use `--save-plot-path <dir>` to save plots to a custom location.

Closed-Loop Evaluation

Test your model in simulation or on real hardware using the server-client architecture:

```
# Start the policy server
uv run python gr00t/eval/run_gr00t_server.py \
  --embodiment-tag NEW_EMBODIMENT \
  --model-path <CHECKPOINT_PATH> \
  --device cuda:0 \
  --host 0.0.0.0 --port 5555
```



```
from gr00t.policy.server_client import PolicyClient

policy = PolicyClient(host="localhost", port=5555)
env = YourEnvironment()
obs, info = env.reset()
action, info = policy.get_action(obs)
obs, reward, done, truncated, info = env.step(action)
```



Debugging with ReplayPolicy: To verify your environment setup without a trained model, start the server with `--dataset-path <DATASET_PATH>` (omit `--model-path`) to replay recorded actions from the dataset.

See the complete [Policy API Guide](#) for observation/action formats, batched inference, and troubleshooting.

Benchmark Examples

We support evaluation on public benchmarks using a server-client architecture. The policy server reuses the project root's uv environment; simulation clients have individual setup scripts.

You can use [the verification script](#) to verify that all dependencies are properly configured.

One-Time Simulation Environment Setup

Each simulation benchmark needs a one-time environment setup before its first run. First install the shared system libraries:

```
sudo apt update
sudo apt install libegl1-mesa-dev libglu1-mesa
```



Then run the benchmark's own `setup_*.sh` script, linked from each simulation benchmark's README (LIBERO, SimplerEnv, robocasa, and robocasa-gr1). This only needs to run once per benchmark; afterward you just launch the server and client. The real-hardware/custom-embodiment workflows (DROID, RoboLab, SO100) have no simulation setup script; follow their own READMEs instead.

Zero-shot (evaluate with the base model, no finetuning):

- [DROID](#) — real-world DROID robot (also available as the finetuned `nvidia/GR00T-N1.7-DROID` checkpoint; `examples/DROID/README.md` covers both paths)

Finetuned (evaluate with finetuned checkpoints):

- [DROID](#) — real-world DROID robot via `nvidia/GR00T-N1.7-DROID`
- [RoboLab](#) — RoboLab simulation tasks via `nvidia/GR00T-N1.7-DROID`
- [LIBERO](#) — LIBERO benchmark (Franka Panda)
- [SimplerEnv](#) — Google Robot (Fractal) and WidowX (Bridge)
- [SO100](#) — SO100 custom embodiment workflow

► Adding a New Sim Benchmark

Running Tests

Install the development dependencies before running the test suite:

```
uv sync --python 3.12 --extra dev
uv run python -m pytest
```



Use targeted test paths for faster local checks, and reserve GPU-marked tests for machines with the required CUDA hardware.

Contributions

We welcome issues and pull requests. See [CONTRIBUTING.md](#) for how to contribute and for support details now that GR00T N1.7 has reached General Availability (GA).

License

- **Code:** Apache 2.0 — see [LICENSE](#)
- **Model weights:** [NVIDIA Open Model License](#)

```
# SPDX-FileCopyrightText: Copyright (c) 2026 NVIDIA CORPORATION & AFFILIATES. All rights reserved.
# SPDX-License-Identifier: Apache-2.0
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```



See the License for the specific language governing permissions and
limitations under the License.

Citation

[Paper Site](#)

```
@inproceedings{gr00tn1_2025,  
  archivePrefix = {arxiv},  
  eprint       = {2503.14734},  
  title        = {{GR00T} {N1}: An Open Foundation Model for Generalist Humanoid Robots},
```



Releases 5

n1.7 release Latest
on Apr 18

[+ 4 releases](#)

Packages

No packages published

Contributors 45



[+ 31 contributors](#)

Languages

